

## Deploy Java App to Minikube Automated with Jenkins

### 1. Overview

Automating the deployment of a Java application to Minikube using Jenkins involves building the application, creating a Docker image, pushing it to a container registry, and deploying it to Minikube using Kubernetes manifests.

---

### 2. Key Concepts

#### A. Jenkins Pipeline

Jenkins automates the CI/CD process using a declarative pipeline. The pipeline consists of multiple stages such as:

- **SCM Checkout:** Fetches code from a repository (GitHub/GitLab).
  - **Build & Test:** Uses Maven (mvn package) to compile and test the Java application.
  - **Docker Build & Push:** Builds a Docker image of the application and pushes it to Docker Hub.
  - **Deploy to Minikube:** Uses kubectl to apply Kubernetes deployment and service files.
- 

#### B. Minikube

Minikube is a lightweight Kubernetes cluster for local development and testing. It allows developers to run Kubernetes locally and deploy applications without needing a cloud-based cluster.

#### Commands to Start Minikube:

```
sh
```

```
CopyEdit
```

```
minikube start
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

---

### **C. Docker**

Docker is used to package the Java application into a container image, making it portable and easy to deploy across environments.

#### **Dockerfile Example:**

```
dockerfile
```

```
CopyEdit
```

```
FROM openjdk:11
```

```
COPY target/webapp.jar /app/webapp.jar
```

```
WORKDIR /app
```

```
CMD ["java", "-jar", "webapp.jar"]
```

---

### **D. Kubernetes Deployment**

Kubernetes YAML files define how the application should be deployed inside the Minikube cluster.

#### **Deployment YAML Example:**

```
yaml
```

```
CopyEdit
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: webapp
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

matchLabels:

app: webapp

template:

metadata:

labels:

app: webapp

spec:

containers:

- name: webapp

image: nidharsan8008/war-jenkins1

ports:

- containerPort: 8080

### **Apply Deployment:**

sh

CopyEdit

kubectl apply -f deployment.yml

kubectl get pods

pipeline {

agent any

stages {

stage('Clone Git Repository') {

steps {

git 'https://github.com/NIDHARSAN-V/Devops\_Guvi\_WAR.git'

```
    }  
}
```

```
stage('Build WAR File') {  
    steps {  
        sh 'mvn clean package' // Build WAR file using Maven  
    }  
}
```

```
stage('Build Docker Image') {  
    steps {  
        script {  
            sh 'docker build -t warimage-jenkins .'  
            sh 'docker tag warimage-jenkins nidharsan8008/warimage-jenkins1'  
        }  
    }  
}
```

```
// stage('Run Docker Container') { // Optional stage to test the container  
//     steps {  
//         script {  
//             sh 'docker run -d -p 8888:8080 --name war-container-jenkins warimage-jenkins'  
//         }  
//     }  
// }  
// }
```

```
stage('Push to Docker Hub') {  
    steps {  
        script {  
            withDockerRegistry(credentialsId: 'dockerhub_token', url:  
'https://index.docker.io/v1/') {  
                sh 'docker push nidharsan8008/warimage-jenkins1'  
            }  
        }  
    }  
}
```

```
stage('Deploy with deployment.yml') {  
    steps {  
        withKubeConfig(caCertificate: '', clusterName: 'minikube ', contextName: 'minikube',  
credentialsId: 'MiniKube_ID', namespace: '', restrictKubeConfigAccess: false, serverUrl:  
'https://192.168.49.2:8443') {  
            sh 'kubectl apply -f deployment.yml'  
        }  
    }  
}
```

```
// stage('Test') {
```

```
//     steps {
```

```
    //      withKubeConfig(caCertificate: "", clusterName: 'minikube ', contextName: 'minikube',
credentialsId: 'MiniKube_ID', namespace: "", restrictKubeConfigAccess: false, serverUrl:
'https://192.168.49.2:32274') {
```

```
        //      sh "
```

```
        //    }
```

```
    // }
```

```
    //}
```

```
}
```

```
}
```



# Terraform :

*#terraform init*

*#terraform validate*

*#terraform plan*

*#terraform apply*

*#terraform destroy*

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "5.92.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "us-east-1"  
}
```

```
resource "aws_vpc" "myvpc" {
```



```
cidr_block    = "10.0.0.0/16"
```

```
tags = {
```

```
    Name = "demovpc"
```

```
}
```

```
}
```

```
resource "aws_subnet" "pubsub" {
```

```
    vpc_id    = aws_vpc.myvpc.id
```

```
    cidr_block = "10.0.1.0/24"
```

```
    availability_zone = "us-east-1a"
```

```
tags = {
```

```
    Name = "sn1"
```

```
}
```

```
}
```

```
resource "aws_subnet" "pub_sub" {
```

```
    vpc_id    = aws_vpc.myvpc.id
```

```
    cidr_block = "10.0.2.0/24"
```

```
    availability_zone = "us-east-1a"
```

```
tags = {
```

```
    Name = "sn1"
  }
}

resource "aws_subnet" "prisub" {
  vpc_id    = aws_vpc.myvpc.id
  cidr_block = "10.0.3.0/24"
  availability_zone = "us-east-1a"

  tags = {
    Name = "sn1"
  }
}
```

```
resource "aws_subnet" "pri_sub" {
  vpc_id    = aws_vpc.myvpc.id
  cidr_block = "10.0.4.0/24"
  availability_zone = "us-east-1a"

  tags = {
    Name = "sn1"
  }
}
```

```
resource "aws_internet_gateway" "tfigw" {
```

```
  vpc_id = aws_vpc.myvpc.id
```

```
  tags = {
```

```
    Name = "tfigw"
```

```
  }
```

```
}
```

```
resource "aws_route_table" "tfpubrt" {
```

```
  vpc_id = aws_vpc.myvpc.id
```

```
  route {
```

```
    cidr_block = "0.0.0.0/0"
```

```
    gateway_id = aws_internet_gateway.tfigw.id
```

```
  }
```

```
  tags = {
```

```
    Name = "tfpublicroute"
```

```
  }
```

```
}
```

```
resource "aws_route_table_association" "pubsn1" {
```

```
    subnet_id    = aws_subnet.pubsub.id
    route_table_id = aws_route_table.tfpubrt.id
}
```

```
resource "aws_route_table_association" "pubsn2" {
    subnet_id    = aws_subnet.pub_sub.id
    route_table_id = aws_route_table.tfpubrt.id
}
```

```
resource "aws_eip" "tfeip" {
    domain = "vpc"
}
```

```
resource "aws_nat_gateway" "tfnat" {
    allocation_id = aws_eip.tfeip.id
    subnet_id    = aws_subnet.pub_sub.id
}
```

```
tags = {
    Name = "gw NAT"
}
}
```

```
resource "aws_route_table" "tfprint" {
```

```
vpc_id = aws_vpc.myvpc.id
```

```
route {  
  cidr_block = "0.0.0.0/0"  
  gateway_id = aws_nat_gateway.tfnat.id  
}
```

```
tags = {  
  Name = "tfprivateroute"  
}  
}
```

```
resource "aws_route_table_association" "prsn3" {  
  subnet_id    = aws_subnet.prisub.id  
  route_table_id = aws_route_table.tfprirt.id  
}
```

```
resource "aws_route_table_association" "prsn4" {  
  subnet_id    = aws_subnet.pri_sub.id  
  route_table_id = aws_route_table.tfprirt.id  
}
```

```
resource "aws_security_group" "allow_tfsg" {  
  name    = "allow_tfsg"
```

description = "Allow TLS inbound traffic"

vpc\_id = aws\_vpc.myvpc.id

ingress {

description = "HTTPS "

from\_port = 443

to\_port = 443

protocol = "tcp"

cidr\_blocks = ["0.0.0.0/0"]

}

ingress {

description = "HTTP "

from\_port = 80

to\_port = 80

protocol = "tcp"

cidr\_blocks = ["0.0.0.0/0"]

}

ingress {

description = "SSH"

from\_port = 22

to\_port = 22

protocol = "tcp"

cidr\_blocks = ["0.0.0.0/0"]

```
}
```

```
egress {
```

```
  from_port    = 0
```

```
  to_port      = 0
```

```
  protocol     = "-1"
```

```
  cidr_blocks  = ["0.0.0.0/0"]
```

```
}
```

```
tags = {
```

```
  Name = "TfsecurityGroup"
```

```
}
```

```
}
```

```
resource "aws_instance" "pub_ins" {
```

```
  ami          = "ami-0fc5d935ebf8bc3bc"
```

```
  instance_type = "t2.micro"
```

```
  subnet_id    = aws_subnet.pub_sub.id
```

```
  vpc_security_group_ids = [aws_security_group.allow_tfsg.id]
```

```
  key_name      = "saran"
```

```
  associate_public_ip_address = "true"
```

```
}
```

# CheatSheet



the essential

**Terraform**

Cheatsheet

by justin o'connor

>

general commands

get the terraform version

terraform version

download and update root modules

terraform get -update=true

open up a terraform interactive terminal

terraform console

create a dot diagram of terraform dependencies

terraform graph | dot -Tpng > graph.png

format terraform code to HCL standards

terraform fmt

validate terraform code syntax

terraform validate

enable tab auto-completion in the terminal

terraform -install-autocomplete

show information about provider requirements

terraform providers

login and logout of terraform cloud

terraform login and terraform logout

>

workspaces

list the available workspaces

terraform workspace list

create a new workspace

terraform workspace new development

select an existing workspace

terraform workspace select default

>

initilize terraform

initialize terraform in the current working directory

terraform init

skip plugin installation

terraform init -get-plugins=false

force plugin installation from a directory

terraform init -plugin-dir=PATH

upgrade modules and plugins at initialization

terraform init -upgrade

update backend configuration

terraform init -migrate-state -force-copy

skip backend configuration

terraform init -backend=false

use a local backend configuration

terraform init -backend-config=FILE

change state lock timeout (default is zero seconds)

terraform init -lock-timeout=120s

>

plan terraform

produce a plan with diff between code and state

terraform plan

output a plan file for reference during apply

terraform plan -out current.tfplan

output a plan to show effect of terraform destroy

terraform plan -destroy

target a specific resource for deployment

terraform plan -target=ADDRESS

*note that the -target option is also available for the terraform apply and terraform destroy commands.*

>

outputs

list available outputs

terraform output

output a specific value

terraform output NAME

>

apply terraform

apply the current state of terraform code

terraform apply

specify a previously generated plan to apply

terraform apply current.tfplan

enable auto-approval or automation

terraform apply -auto-approve

>

destroy terraform

destroy resources managed by terraform state

terraform destroy

enable auto-approval or automation

terraform destroy -auto-approve

>

manage terraform state

list all resources in terraform state

terraform state list

show details about a specific resource

terraform state show ADDRESS

track an existing resource in state under new name

terraform state mv SOURCE DESTINATION

import a manually created resource into state

terraform state import ADDRESS ID

pull state and save to a local file

terraform state pull > terraform.tfstate

push state to a remote location

terraform state push PATH

replace a resource provider

terraform state replace-provider A B

taint a resource to force redeployment on apply

terraform taint ADDRESS

untaint a previously tainted resource

terraform untaint ADDRESS

Version 1

<https://justinoconnor.codes>